

Ejercicio 1:

Creas una DependencyProperty para cambiar dinámicamente el tamaño del texto de un Button.

1. Creas una DependencyProperty llamada TamañoTexto.
2. Utilizas esta propiedad para modificar el tamaño de fuente de un botón al hacer clic.

SOLUCIÓN:

XAML

```
<Window x:Class="MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Ejercicio 1" Height="200" Width="400">
    <Button Content="Haz clic para cambiar tamaño"
            FontSize="{Binding TamañoTexto}"
            Click="Button_Click"/></Window>
```

C#

```
public partial class MainWindow : Window
{
    // 1. Definir la DependencyProperty
    public static readonly DependencyProperty TamañoTextoProperty =
        DependencyProperty.Register(
            "TamañoTexto", // Nombre de la propiedad
            typeof(double), // Tipo de la propiedad
            typeof(MainWindow), // Tipo de la clase que contiene
                                // la propiedad
            new PropertyMetadata(12.0)); // Valor predeterminado

    // 2. Propiedad pública que envuelve la DependencyProperty
    public double TamañoTexto
    {
        get { return (double)GetValue(TamañoTextoProperty); }
        set { SetValue(TamañoTextoProperty, value); }
    }

    public MainWindow()
    {
        InitializeComponent();
        // 3. Establecer el DataContext de la ventana al propio objeto
        this.DataContext = this; // Esto hace que el DataContext sea la
                                // instancia de MainWindow
    }

    private void Button_Click(object sender, RoutedEventArgs e)
    {
        // 4. Aumentar el tamaño del texto al hacer clic
        TamañoTexto += 2.0; // Cambiar el valor de la propiedad TamañoTexto
    }
}
```

Ejercicio 2:

Aplicar propiedades de dependencia con plantillas para cambiar dinámicamente el diseño de un control.

1. Diseña una ventana con un TextBox y un botón.
2. Configura una propiedad de dependencia llamada ColorDeFondoTexto que permita cambiar el color de fondo del TextBox cuando el botón es presionado.
3. Crea la clase ColorDeFondoTexto que herede de la clase Brush.
4. Usa un estilo en XAML que aplique dinámicamente esta propiedad con triggers.

XAML:

```
<Window x:Class="Ejemplo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Propiedades de Dependencia" Height="200" Width="400">
    <StackPanel Margin="10">
        <TextBox Name="MiTextBox" Text="Escribe algo aquí..." Height="30" />
        <Button      Content="Cambiar      Fondo"      Height="30"      Margin="10"
Click="CambiarFondo_Click" />
    </StackPanel>
</Window>
```

C#

```
using System.Windows;
using System.Windows.Media;

namespace Ejemplo
{
    public partial class MainWindow : Window
    {
        public static readonly DependencyProperty ColorDeFondoTextoProperty =
            DependencyProperty.Register(
                "ColorDeFondoTexto",
                typeof(Brush),
                typeof(MainWindow),
                new PropertyMetadata(Brushes.White)
            );
    }
}
```

```
);

public Brush ColorDeFondoTexto
{
    get { return (Brush)GetValue(ColorDeFondoTextoProperty); }
    set { SetValue(ColorDeFondoTextoProperty, value); }
}

public MainWindow()
{
    InitializeComponent();
    MiTextBox.Background = ColorDeFondoTexto;
}

private void CambiarFondo_Click(object sender, RoutedEventArgs e)
{
    ColorDeFondoTexto = ColorDeFondoTexto == Brushes.Yellow ? Brushes.White :
Brushes.Yellow;
    MiTextBox.Background = ColorDeFondoTexto;
}
}
```

[Ejercicio 3: Enlace de datos TwoWay con TextBox y Button](#)

Crea un enlace entre un TextBox y una propiedad de dependencia. El contenido del TextBox se reflejará en un Button.

1. Define una propiedad de dependencia llamada TextoUsuario.
2. Usa esta propiedad para mostrar y modificar el contenido en un TextBox.
3. Usa esta misma propiedad para cambiar dinámicamente el contenido de un Button.
4. Englóballo todo en un StackPanel.

Comportamiento: cuando escribas en el TextBox, al pulsar el botón, deberá igualarse el contenido del botón a lo que se haya escrito en el TextBox.

SOLUCIÓN:

XAML

```
<StackPanel>

    <!-- Enlace del TextBox a la propiedad de dependencia TextoUsuario -->

    <TextBox      Name="textoUsuarioTextBox"      Text="{Binding      TextoUsuario,
UpdateSourceTrigger=PropertyChanged}" Margin="10" Width="200"/>


    <!-- Enlace del Button a la propiedad de dependencia TextoUsuario -->

    <Button  Content="{Binding  TextoUsuario}"  Click="Button_Click"  Margin="10"
Width="200"/>
```

C#

```
public partial class MainWindow : Window
{
    // Definir la propiedad de dependencia TextoUsuario
    public static readonly DependencyProperty TextoUsuarioProperty =
        DependencyProperty.Register("TextoUsuario",                typeof(string),
typeof(MainWindow), new PropertyMetadata(string.Empty));


    // Propiedad para acceder a la propiedad de dependencia
    public string TextoUsuario
    {
        get { return (string)GetValue(TextoUsuarioProperty); }
        set { SetValue(TextoUsuarioProperty, value); }
    }


    public MainWindow()
    {
        InitializeComponent();

        // Establecer el DataContext a la instancia de la ventana para el enlace
        DataContext = this;
    }
}
```

```
private void Button_Click(object sender, RoutedEventArgs e)
{
    // Al hacer clic, actualizar el valor de la propiedad de dependencia
    TextoUsuario = textoUsuarioTextBox.Text;
}
}
```

Ejercicio 4:

Modifica el ejercicio anterior para que se cambie automáticamente el contenido del botón al escribir en el TextBox, para ello recurre a la propiedad: UpdateSourceTrigger.

SOLUCIÓN:

XAML

```
<StackPanel>
    <!-- TextBox para modificar la propiedad -->
    <TextBox Width="200"
        Text="{Binding TextoUsuario, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"
        Margin="10"/>
    <!-- Button que refleja el texto -->
    <Button Width="200"
        Content="{Binding TextoUsuario, Mode=TwoWay}" />
</StackPanel>
```

C#

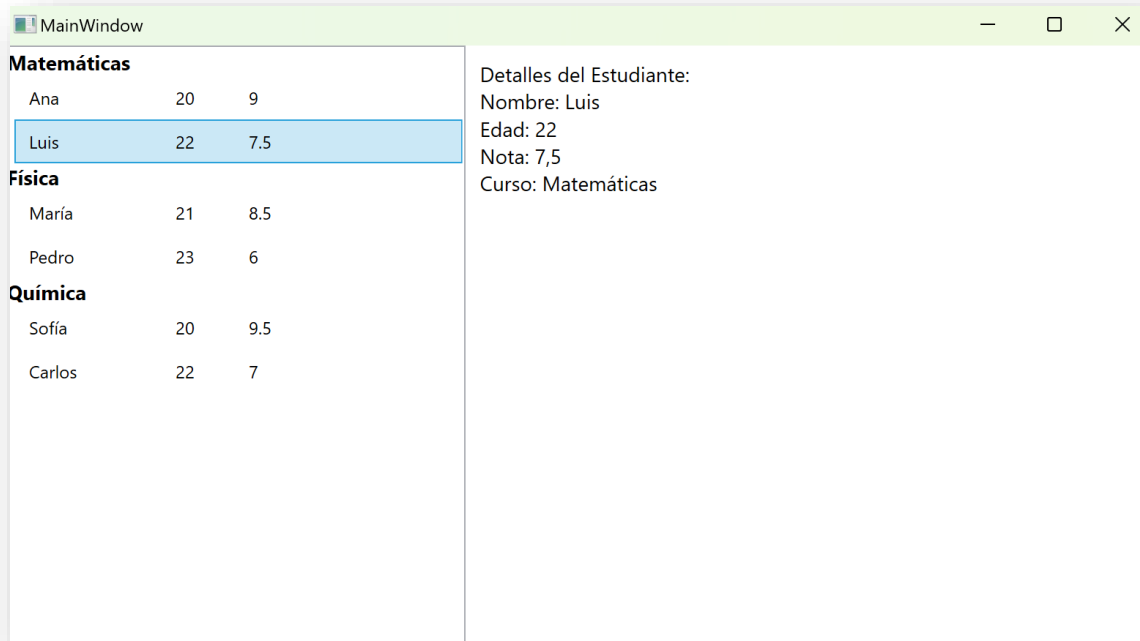
Ejercicio 5: ListBox con agrupación y visualización de detalles

Crear una aplicación WPF que muestre una lista de estudiantes agrupados por curso. Cada estudiante tendrá propiedades como **Nombre**, **Edad**, **Nota** y **Curso**. Al seleccionar un estudiante en la lista, se deben mostrar sus detalles en un TextBlock en otra sección de la ventana.

Deberá verse así:

CFGS Desarrollo de Aplicaciones Multiplataforma

Módulo: Desarrollo de Interfaces



SOLUCIÓN 1: Usando la clase ListCollectionView

XAML

```
<Grid>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="2*" />
        <ColumnDefinition Width="3*" />
    </Grid.ColumnDefinitions>

    <!-- ListBox con agrupación -->
    <ListBox Name="lbEstudiantes" Grid.Column="0"
        SelectionChanged="lbEstudiantes_SelectionChanged">
        <ListBox.GroupStyle>
            <GroupStyle>
                <GroupStyle.HeaderTemplate>
                    <DataTemplate>
                        <TextBlock FontWeight="Bold" Text="{Binding Name}"
FontSize="14" />
                    </DataTemplate>
                </GroupStyle.HeaderTemplate>
            </GroupStyle>
        </ListBox.GroupStyle>

        <!-- Plantilla para mostrar estudiantes -->
        <ListBox.ItemTemplate>
            <DataTemplate>
                <StackPanel Orientation="Horizontal" Margin="5">
                    <TextBlock Text="{Binding Nombre}" Width="100" />
                    <TextBlock Text="{Binding Edad}" Width="50"
HorizontalAlignment="Right"/>
                    <TextBlock Text="{Binding Nota}" Width="50"
HorizontalAlignment="Right" />
                </StackPanel>
            </DataTemplate>
        </ListBox.ItemTemplate>
    </ListBox>
```

```
<!-- TextBlock para mostrar detalles -->
<TextBlock Name="tbDetalles" Grid.Column="1" FontSize="14"
TextWrapping="Wrap" Margin="10" />
</Grid>
```

C#

```
public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();

        var estudiantes = new ListCollectionView(new List<Estudiante>
        {
            new Estudiante { Nombre = "Ana", Edad = 20, Nota = 9.0, Curso =
"Matemáticas" },
            new Estudiante { Nombre = "Luis", Edad = 22, Nota = 7.5, Curso =
"Matemáticas" },
            new Estudiante { Nombre = "María", Edad = 21, Nota = 8.5, Curso =
"Física" },
            new Estudiante { Nombre = "Pedro", Edad = 23, Nota = 6.0, Curso =
"Física" },
            new Estudiante { Nombre = "Sofía", Edad = 20, Nota = 9.5, Curso =
"Química" },
            new Estudiante { Nombre = "Carlos", Edad = 22, Nota = 7.0, Curso =
"Química" },
        });

        // Agrupación por curso
        estudiantes.GroupDescriptions.Add(new
PropertyGroupDescription("Curso"));

        // Enlazar al ListBox
        lbEstudiantes.ItemsSource = estudiantes;
    }
    private void lbEstudiantes_SelectionChanged(object sender,
SelectionChangedEventArgs e)
    {
        if (lbEstudiantes.SelectedItem is Estudiante estudianteSeleccionado)
        {
            tbDetalles.Text = $"Detalles del Estudiante:\n" +
                $"Nombre: {estudianteSeleccionado.Nombre}\n" +
                $"Edad: {estudianteSeleccionado.Edad}\n" +
                $"Nota: {estudianteSeleccionado.Nota}\n" +
                $"Curso: {estudianteSeleccionado.Curso}";
        }
    }
}
```

SOLUCIÓN 2: Haciendo una lista simple

XAML

```
<Grid>

    <Grid.ColumnDefinitions>

        <ColumnDefinition Width="2*" />

        <ColumnDefinition Width="3*" />
```

CFGS Desarrollo de Aplicaciones Multiplataforma
Módulo: Desarrollo de Interfaces

```
</Grid.ColumnDefinitions>

<!-- ListBox enlazado al CollectionViewSource -->
<ListBox ItemsSource="{Binding Source={StaticResource EstudiantesView}}"
Grid.Column="0"
SelectionChanged="lbEstudiantes_SelectionChanged">
  <ListBox.GroupStyle>
    <GroupStyle>
      <GroupStyle.HeaderTemplate>
        <DataTemplate>
          <TextBlock FontWeight="Bold" Text="{Binding Name}" FontSize="14" />
        </DataTemplate>
      </GroupStyle.HeaderTemplate>
    </GroupStyle>
  </ListBox.GroupStyle>

  <!-- Plantilla para mostrar estudiantes -->
  <ListBox.ItemTemplate>
    <DataTemplate>
      <StackPanel Orientation="Horizontal" Margin="5">
        <TextBlock Text="{Binding Nombre}" Width="100" />
        <TextBlock Text="{Binding Edad}" Width="50"
HorizontalAlignment="Right"/>
        <TextBlock Text="{Binding Nota}" Width="50"
HorizontalAlignment="Right" />
      </StackPanel>
    </DataTemplate>
  </ListBox.ItemTemplate>
</ListBox>

<!-- TextBlock para mostrar detalles -->
```


CFG5 Desarrollo de Aplicaciones Multiplataforma
Módulo: Desarrollo de Interfaces

```
<TextBlock      Name="tbDetalles"      Grid.Column="1"      FontSize="14"
TextWrapping="Wrap" Margin="10" />
</Grid>
```

C#

```
public partial class MainWindow : Window
{
    public List<Estudiante> Estudiantes { get; set; }

    public MainWindow()
    {
        InitializeComponent();
        Estudiantes = new List<Estudiante>
        {
            new Estudiante { Nombre = "Ana", Edad = 20, Nota = 9.0, Curso = "Matemáticas"
        },
            new Estudiante { Nombre = "Luis", Edad = 22, Nota = 7.5, Curso = "Matemáticas"
        },
            new Estudiante { Nombre = "María", Edad = 21, Nota = 8.5, Curso = "Física" },
            new Estudiante { Nombre = "Pedro", Edad = 23, Nota = 6.0, Curso = "Física" },
            new Estudiante { Nombre = "Sofía", Edad = 20, Nota = 9.5, Curso = "Química" },
            new Estudiante { Nombre = "Carlos", Edad = 22, Nota = 7.0, Curso = "Química" }
        };

        // Enlazamos la lista básica al DataContext
        DataContext = this;
    }
}

// Clase Estudiante
public class Estudiante
{
    public string Nombre { get; set; }
```

CFGS Desarrollo de Aplicaciones Multiplataforma

Módulo: Desarrollo de Interfaces

```
public int Edad { get; set; }  
public double Nota { get; set; }  
public string Curso { get; set; }  
}
```